

Individual Contribution Reflection

Tom Tian (540180866)

COMP4347 / COMP5347 Assignment 2 – MERN Quiz Game (variation: Review Mode after completion)

Subsystem D: System Integration, Validation & Robustness (group-of-four mandated role, spec §11.1)

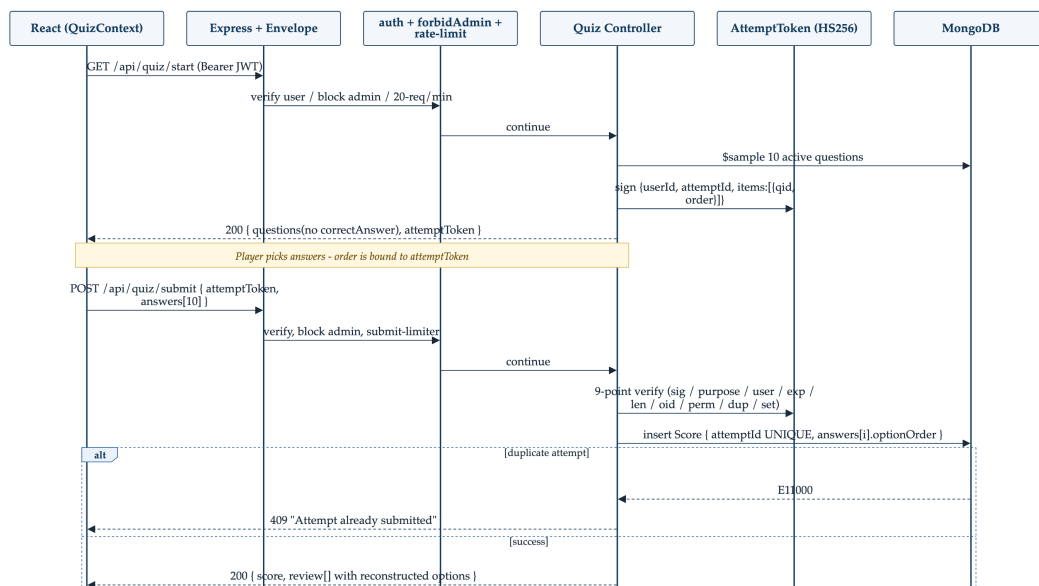
1. Subsystem ownership. I owned Subsystem D, the cross-cutting role that the spec reserves for one student in a four-person group. I was not the primary author of any feature slice (Auth/Tracy, Quiz Core/Raven, Admin CRUD/Allen). My work was the wiring that let those independently developed slices compose into one application without contract drift. Scope: (a) the shared `{success,data?,error?}` envelope (`utils/responseEnvelope.js`) and the centralised error handler that sanitises 5xx leakage (`middleware/errorHandler.js:21-27`); (b) the signed `attemptToken` integrity layer (§2); (c) global hardening with `helmet`, `express-mongo-sanitize`, a CORS allow-list, a 1 MB body cap, and per-user rate limiters keyed on `req.user?.id || req.ip`; (d) Swagger UI at `/api-docs` plus a mirrored Postman collection so reviewers can verify the API in either tool; (e) the React shell, `ThemeContext`, `ProtectedRoute/ProtectedAdminRoute`, and the `ActiveQuizNavigationGuard` that intercepts refresh, popstate, and link-clicks while a quiz is active (`frontend/src/App.jsx:28-1`); (f) 6 backend and 4 frontend test suites (**29 + 13 passing**) that lock the cross-subsystem contracts; (g) an idempotent Mongo seed and a root `npm run demo` one-command bootstrap. 18 source files start with a `// Subsystem D - Integration, Robustness & Documentation (Tom Tian):` header so ownership is grep-able.

2. Major technical challenge: securing the shuffled-attempt contract. Review Mode forces an end-to-end ordering chain. `/start` must shuffle options per attempt (so a player cannot memorise indices), `/submit` must score against the original `correctAnswer` without ever leaking it to the client, and `/history/:id` must rebuild the exact options the player saw. A naive design (trusting client-sent indices, or echoing `correctAnswer` back at `/start`) breaks scoring integrity *and* Review Mode at the same time. While stabilising the test suite I noticed it was hiding contract drift by hardcoding `correctAnswer === 0` (fixed in `bba8b06`). My fix was to make the server the only source of truth: a JWT `attemptToken` is signed at `/start` carrying the per-question option permutation, the exact question IDs, a user binding, and a `crypto.randomUUID()` attempt ID; the same permutation is persisted as `Score.answers[i].optionOrder` so any later review page renders identically (`73d24c6`). The verifier rejects *nine* failure modes: bad signature, wrong purpose claim, user mismatch, expiry, wrong length, malformed `ObjectId`, non-permutation order, duplicate qids, and qid-set mismatch (`backend/src/utils/quizAttemptToken.js:60-101`). Replay protection runs in *four* layers: a frontend `submitOnceRef`, a controller pre-check, a MongoDB `err.code === 11000` catch, and a unique partial index on `Score.attemptId`. Scoring uses a reverse-map, so the answer key never leaves the server:

```
// backend/src/utils/quizAttemptToken.js:45-57 -- signed at /start
const attemptId = crypto.randomUUID();
const payload = {
  purpose: 'quiz_attempt', userId: String(userId), attemptId,
  items: questions.map(q => ({ qid: String(q._id), order: q.optionOrder })),
};
const token = jwt.sign(payload, getJwtSecret(), { expiresIn: ATTEMPT_TOKEN_TTL_SECONDS });

// backend/src/controllers/quiz.controller.js:149-151 -- reverse-mapped scoring at /submit
const originalIndex = item.order[ans.selectedAnswer]; // visible idx -> stored idx
const isCorrect = originalIndex === question.correctAnswer;
```

3. Sequence diagram for a secured Review Mode attempt (Mermaid source rendered to PNG with `mmdc`; source in `.tools/reflection-build/`).



4. Variation design decisions (Review Mode). Three integration decisions shaped how Subsystem D interacted with the variation. (i) *Persist `optionOrder` per answer instead of a frozen snapshot of the question text.* A full snapshot would lock typos into history, would force admins to rewrite past attempts every time they edit a question, and would make every `Score` document significantly larger. Persisting only the 4-element permutation keeps the

review faithful while the live Question stays authoritative; the rare deleted-question case is handled with a [Question deleted] placeholder (quiz.controller.js:271-282). (ii) Strip correctAnswer at /start and score by reverse-mapping the visible index. toStartQuizPayload (utils/shuffleQuestion.js:55-62) removes the field, and submit compares item.order[selected] === question.correctAnswer. Because the client never receives the answer key, client tampering cannot change the score. (iii) Generate attempts dynamically per Ed #182, with no Quiz collection. The signed attemptToken is the only thing that crosses the start→submit boundary, so no quiz state is persisted before submission, half-finished attempts cost no DB storage, and the spec’s “answer cannot be changed once submitted” rule is enforced by token expiry rather than by session bookkeeping.

5. Git commit history analysis. 35 non-merge commits on origin/final across two aliases (Tom Tian = 32; early Tom_Tian = 3; both documented in README.md “Git Workflow”). 14 representative commits below, each linking to a commit on Sydney GitHub Enterprise.

Hash	Focus	Technical evidence (what changed + why it matters)
7a61d82	Project bootstrap	Established the monorepo layout, root scripts (install:all, dev, seed, demo), .env.example files, and the initial .gitignore. Sets up the install/run path used during marking.
eb118b5	Shared React app shell	Added React Router shell, AppShellNavbar that swaps Player/Admin/Public navbars by route, and the ProtectedRoute/ProtectedAdminRoute wrappers consumed by every page.
6686eae	Seed + integration tests	Idempotent seeds/seed.js (1 admin + 2 players + 10 active questions; safe to re-run) and tests/api.test.js for the auth + quiz happy path.
972fc77	D-owned QA alignment	Reconciled validators.test.js / errorHandler.test.js / envelope.test.js with the delivered controller behaviour; removed stale fixtures that were masking real diffs.
160ac65	Risk note (no code crossover)	Documented a leaderboard/auth QA risk in writing only, deliberately leaving A/B code untouched. Keeps the local-ownership boundary agreed to in AGENTS.md.
bba8b06	Shuffled-answer test lock	Rewrote api.test.js to derive the visible index from the live /start response instead of asserting correctAnswer === 0. Closes the test blind spot that motivated \$2.
4a8fe22	Swagger / Postman alignment	Synced the leaderboard and auth OpenAPI annotations with the actual response envelope, including the 401/429 negative paths, so the two API doc sets stay in sync.
d0d07f6	Quiz error catalogue	Documented every 400/401/403/409/429 path on /api/quiz/* in Swagger so the negative-path contract can be read without opening the controller.
73d24c6	Signed attempt integrity	Introduced quizAttemptToken.js, the 9-check verifier, the Score.attemptId unique partial index, and a frontend submitOnceRef. Added test cases for tampered, wrong-user, replayed, and expired tokens. The main robustness change.
fd08cc8	Admin ↔ player boundary	Added the forbidAdminQuiz middleware and the frontend <ProtectedRoute blockAdmin>. Admin JWTs now receive 403 on /api/quiz/*, and players are redirected away from /admin; symmetric access control on both layers.
06697e1	Active-quiz navigation guard	ActiveQuizNavigationGuard intercepts beforeunload, popstate, and document-level link clicks while hasActiveQuiz === true. Prevents accidental progress loss and the refresh-until-easy-question pattern.
a6e4ad7	Rate-limit split + API root	Separated registerLimiter from loginLimiter so the register endpoint cannot be starved by login probing. Exposed /api root with version metadata.
24ad7e7	Unified “new run” UX	Centralised the post-quiz “play again” reducer action so Result, History, and Review pages all trigger the same QUIZ_RESET/QUIZ_ERROR_RESET flow. Removed three small context-state bugs at the same time.
f5d3d98	Final reverse-mapping lock	Added a regression test that constructs 10 deliberately-wrong answers, submits them, and asserts score === 0; locks the reverse-mapping invariant before the final merge to final.

6. Reflection ↔ Spec §11 compliance map. Every rubric-mandated element of the Individual Reflection is covered above; this table is the index to where.

Spec §11 mandate	Where	Concrete evidence anchor
Subsystem responsibility + how implemented	\$1	7 itemised scope buckets (a)–(g) anchored to file paths, test counts, and middleware names.
≥1 major technical challenge with code snippets or commit links	\$2	9-line excerpt from quizAttemptToken.js:45-57 + quiz.controller.js:149-151; commits bba8b06, 73d24c6.
Mermaid sequence diagram or UML diagram	\$3	Rendered Mermaid PNG (mmdc + custom CSS): 6 participants, 18 numbered messages, alt/else branch for 409-replay vs 200-success.
Git history analysis: 12–15 meaningful commits, hashes + explanations	\$5	14-row table with linked hashes; total corpus = 35 non-merge commits on origin/final.
Variation design-decision reflection	\$4	3 numbered Review Mode decisions, each stating the rejected alternative and the reason.
Group-of-4 integration role (§11.1)	Header + \$1	Subtitle cites “group-of-four mandated role, spec §11.1” verbatim.

7. Ownership boundary (per the team’s AGENTS.md local-ownership rule: Subsystem D does not silently edit A/B/C code).

Tier	Items
Owned (primary)	Response envelope + central error handler; signed attemptToken + 9-check verifier; Score.attemptId unique partial index; rate-limiters + hardening middleware (helmet / mongo-sanitize / CORS / body cap); React shell + ProtectedRoute/ProtectedAdminRoute + ActiveQuizNavigationGuard + ThemeContext; Swagger UI + Postman collection; 6 backend + 4 frontend test suites; idempotent seed + npm run demo.
Supported	Quiz-submit validation paths; forbidAdminQuiz boundary; option-order persistence on Score.answers; bulk-import per-row error envelope; Swagger annotations for routes I did not author (commits fd08cc8, d0d07f6, 4a8fe22).
Not claimed	A/Tracy: auth UI, JWT issuance, bcrypt hashing. B/Raven: quiz screens, scoring controller, Review/History/Leaderboard UI. C/Allen: admin CRUD controller and bulk-import handler logic.

8. Cross-system readiness and final reflection (spec §14, bus factor). Because Subsystem D wires every feature, I can answer demo questions on the full stack: auth (JWT lifecycle, bcrypt, generic invalid-credential message, 5/min login limiter); quiz core (\$sample 10, Fisher–Yates, attempt-token verify, reverse-mapped scoring); admin (authMiddleware + adminMiddleware, bulk-import error reporting); review/history (populate() with optionOrder reconstruction and the [Question deleted] fallback); leaderboard (best-per-user, earliest-timestamp tie-break, per Ed #175). At submission time, npm test is 29/29 backend and 13/13 frontend, npm run build is green, and npm audit -omit=dev is clean. *Final reflection.* For an integration role, what really matters is whether the four feature streams merge cleanly at submission time. Three decisions made the difference here: choosing the signed-token approach instead of trusting client-sent indices, persisting optionOrder instead of snapshotting the question text, and writing the negative-path tests before the contract drifted.